

1 Метаданные и цели

Цель работы: Изучить принципы работы документоориентированной СУБД Apache CouchDB. Освоить взаимодействие с базой данных посредством архитектуры REST API, изучить механизм версионирования документов (MVCC) для разрешения конфликтов и применить концепцию Schema-free совместно с технологией MapReduce для фильтрации данных.

2 Задача 2. Создание базы данных и CRUD-операции

Для взаимодействия с CouchDB использовалась утилита командной строки `curl`. На первом этапе была успешно создана база данных `iot_data` (метод PUT).

```
C:\Users\Nikita>curl -X PUT http://admin:adminadmin@127.0.0.1:5984/iot_data
{"ok":true}
```

Рис. 1: Создание базы данных `iot_data`

Далее был создан первый документ, описывающий показания датчика S1. Сервер вернул сгенерированный `id` и начальную ревизию `_rev`.

```
C:\Users\Nikita>curl -X POST http://admin:adminadmin@127.0.0.1:5984/iot_data -H "Content-Type: application/json" -d "{\"sensor\": \"S1\", \"temperature\": 22.5}"
{"ok":true,"id":"69e81da6db4a771cf86e6a5834000a04","rev":"1-f3b7dc01ab95ec395b1775760d46763f"}
```

Рис. 2: Создание документа (датчик S1, метод POST)

Созданный документ был успешно прочитан по его идентификатору с помощью метода GET.

```
C:\Users\Nikita>curl -X GET http://admin:adminadmin@127.0.0.1:5984/iot_data/69e81da6db4a771cf86e6a5834000a04
{"_id":"69e81da6db4a771cf86e6a5834000a04","_rev":"1-f3b7dc01ab95ec395b1775760d46763f","sensor":"S1","temperature":22.5}
```

Рис. 3: Чтение созданного документа (метод GET)

3 Задача 3. Исследование конфликтов (MVCC)

Была произведена попытка обновления температуры в документе на значение 25.0 без указания параметра ревизии `_rev`. База данных заблокировала операцию, выдав ошибку 409 Conflict, что демонстрирует работу оптимистичной блокировки.

```
C:\Users\Nikita>curl -X PUT http://admin:adminadmin@127.0.0.1:5984/iot_data/69e81da6db4a771cf86e6a5834000a04 -H
"Content-Type: application/json" -d "{\"sensor\": \"S1\", \"temperature\": 25.0}"
{"error":"conflict","reason":"Document update conflict."}
```

Рис. 4: Воспроизведение ошибки 409 Conflict (попытка обновления без `_rev`)

После добавления системного поля `_rev` с актуальным хэшем ревизии в тело JSON-запроса, операция обновления (PUT) завершилась успешно (статус 201 Created). База данных вернула новую ревизию документа.

```
C:\Users\Nikita>curl -X PUT http://admin:adminadmin@127.0.0.1:5984/iot_data/69e81da6db4a771cf86e6a5834000a04 -H "Content-Type: application/json" -d '{"sensor": "\S1", "temperature": 25.0, "_rev": "\1-f3b7dc01ab95ec395b1775760d46763f\"}' {"ok":true,"id":"69e81da6db4a771cf86e6a5834000a04","rev":"2-fb6db19dcc8cf5da4a90924d80a8d703"}
```

Рис. 5: Успешное обновление документа с передачей актуального параметра `_rev`

4 Задача 4. Schema-free и создание MapReduce представления

Особенность Schema-free (отсутствие жесткой схемы таблиц) была продемонстрирована путем добавления в единую базу данных двух новых документов с отличной структурой:

- Датчик S2 с дополнительным полем `location`.
- Датчик S3 с дополнительным полем `humidity`.

```
C:\Users\Nikita>curl -X POST http://admin:adminadmin@127.0.0.1:5984/iot_data -H "Content-Type: application/json" -d '{"sensor": "\S2", "temperature": 15.0, "location": "\outdoor\"}' {"ok":true,"id":"69e81da6db4a771cf86e6a5834001763","rev":"1-a97140aee6851cf1a76a18d8a6661cba"}
```

Рис. 6: Добавление документа с новой структурой (датчик S2, поле `location`)

```
C:\Users\Nikita>curl -X POST http://admin:adminadmin@127.0.0.1:5984/iot_data -H "Content-Type: application/json" -d '{"sensor": "\S3", "temperature": 30.5, "humidity": 45}' {"ok":true,"id":"69e81da6db4a771cf86e6a5834001c73","rev":"1-bba52bc26f4417ad4c6501f1e16bb756"}
```

Рис. 7: Добавление документа с новой структурой (датчик S3, поле `humidity`)

Через веб-интерфейс Fauxton был создан Design Document с Map-функцией на языке JavaScript, которая фильтрует документы, оставляя только те, у которых значение поля `temperature > 20`. В результате фильтрации отобразились только датчики S1 (25.0) и S3 (30.5).

id	key	value
69e81da6db4a771cf86e6a5834000a04	S1	25
69e81da6db4a771cf86e6a5834001c73	S3	30.5

Рис. 8: Результат выполнения MapReduce-представления в веб-интерфейсе Fauxton

5 Выводы

В ходе выполнения лабораторной работы были успешно освоены базовые принципы работы с Apache CouchDB.

Объяснение механизма `_rev` и зачем он нужен:

Поле `_rev` (revision) является ключевым элементом механизма MVCC (Multiversion

Concurrency Control — управление конкурентным доступом с помощью многоверсионности). CouchDB не использует традиционные (пессимистичные) блокировки таблиц или строк. Вместо этого применяется оптимистичная стратегия:

1. Каждое изменение документа порождает новую версию (ревизию).
2. Чтобы обновить или удалить документ, клиент обязан передать актуальное значение `_rev`, которое он предварительно прочитал.
3. Если пока первый клиент редактировал данные, второй клиент уже успел сохранить свои изменения (изменив `_rev` в базе), запрос первого клиента будет отклонен сервером с ошибкой `409 Conflict`.

Данный механизм критически важен в распределенных и веб-системах. Он предотвращает проблему «потерянных обновлений» (lost updates), когда несколько пользователей конкурентно изменяют один и тот же документ, гарантируя целостность и согласованность данных.